

BOA

On-Demand Skilled Help Platform

Master Technical & Implementation Document

React Native (Expo) · Supabase · Feature-First Architecture · Liquid Design System

Author	Kwabena Boateng Konadu (Tether)
Project	BOA — Handyman / Roadside Rescue Platform
Version	1.0 — MVP Phase
Stack	Expo React Native + Supabase + Zustand + NativeWind
Target	Kumasi, Ghana (Phase 1 launch city)

Table of Contents

1. Project Vision & Problem Statement

- Core problem
- The product
- Phases overview

2. Technical Stack

- Final library list
- Why each library
- Future additions

3. Application Architecture

- Folder structure
- Feature-first rationale
- Data layer & repository pattern

4. Database Schema

- All tables with fields
- RLS policies
- Future schema additions

5. Design System

- Liquid design philosophy
- Tokens
- Motion configs
- Component library

6. App Flow & Screen Map

- Customer flow (6 screens)
- Worker flow
- Screen-by-screen specs

7. Implementation Plan

- Sprint 1–6 (Phase 1 MVP)
- Sprint 7–9 (Phase 2)
- Sprint 10–11 (Phase 3)

8. Worker Onboarding Strategy

- USSD system
- WhatsApp API
- Trade associations
- Zone agents
- Community radio

9. Africa Mode — Network Resilience

- Retry logic
- Offline queuing

Fallback flows

10. Growth Roadmap

Phase milestones

Monetisation model

Expansion path

SECTION 1

Project Vision & Problem Statement

The Core Problem

In Ghana and across Africa, when something breaks — a leaking sink, a blown fuse, a stalled car — there is no central emergency system. You shout for help, ask neighbours, dig through phone contacts, or just wait. The process is slow, untransparent, and built entirely on informal networks and personal trust.

BOA (Twi: "help") is an on-demand platform that connects people facing urgent skilled labour needs — plumbing, electrical, general repairs — with verified nearby workers, in minutes not hours. The model is Uber for skilled help, adapted for African infrastructure realities: poor network conditions, informal workers without smartphones, and mobile-money as the primary payment rail.

The "Panic Button" insight

Urgency drives adoption. BOA targets situations where people are forced to act — not convenience services they might use someday.

The Product

- Customer app — request help in under 60 seconds
- Worker app — receive and accept nearby jobs
- Admin dashboard — manual dispatch in Phase 1, analytics in Phase 2
- USSD self-registration — workers without smartphones can join via *384*BOA#
- WhatsApp job dispatch — workers receive job alerts without installing the app

Phase Roadmap Overview

Phase	Timeline	Goal	Key Milestones
1 — Validate	0–3 months	Prove the idea works	5–10 real users · manual dispatch · Kumasi only
2 — Automate	3–9 months	Remove manual steps	Auto-matching · worker app · mobile money · admin dashboard
3 — Platform	9 months+	Multi-vertical scale	Roadside vertical · Accra launch · fleet contracts · subscriptions

SECTION 2

Technical Stack

Final Library List — Phase 1 MVP

Library	Purpose	Phase
expo-router	File-based navigation, type-safe, Expo native	Phase 1
react-native-reanimated	60fps spring animations on UI thread	Phase 1
react-native-gesture-handler	Required by reanimated + bottom sheet	Phase 1
@gorhom/bottom-sheet	iOS-style sheet — match found screen lives here	Phase 1
zustand	Lightweight client state — replaces Redux on mobile	Phase 1
@supabase/supabase-js	Auth + DB + Realtime in one client	Phase 1
react-hook-form	Form state without controlled-component hell	Phase 1
zod	Type-safe schema validation — paired with forms	Phase 1
expo-location	GPS auto-fill on describe screen — CRITICAL	Phase 1
expo-image-picker	Photo upload on describe screen — CRITICAL	Phase 1
expo-notifications	Push notification delivery to workers	Phase 1
expo-haptics	Tactile feedback on key interactions	Phase 1
expo-blur	iOS liquid glass surface — use carefully on Android	Phase 1
react-native-maps	Google Maps SDK — tracking + match found map	Phase 1
@expo/vector-icons	Zero-config icon library bundled with Expo	Phase 1
nativewind	Tailwind CSS for React Native	Phase 1

Android blur warning

expo-blur degrades on mid-range Android devices. Test on a 4GB RAM Android before committing blur to core components. Use sp

Libraries Deferred to futureAdditions.md

Do not install these during Phase 1. Document them in futureAdditions.md and revisit when the trigger condition is met.

Node.js / Express backend	When payment webhooks need a non-Supabase handler (Phase 2)
BullMQ (Redis queue)	When manual dispatch cannot keep up with volume
Socket.IO	Replace Supabase realtime when multi-server scale is needed
@tanstack/react-query	Add when data fetching complexity outgrows Supabase hooks
Tamagui	Consider if NativeWind performance issues appear
AsyncStorage (full offline)	When offline-first becomes a hard requirement

SECTION 3

Application Architecture

Architecture Philosophy — Three Laws

1. Feature-first, not type-first

Each feature owns its own folder — types, repository, store, and hook. No flat /components or /utils bins where everything lives.

2. UI never knows where data comes from

A screen calls a hook. The hook calls a repository. The repository talks to Supabase. Swap Supabase for your own backend by only touching the repository layer.

3. The design system is sacred

Colors, spacing, radius, spring configs — all live in /design. Change once, update everywhere. Never hardcode values in screens.

Data Layer Pattern

The repository pattern is the single most important architectural decision. It means every screen is 3 lines of logic max — no direct Supabase calls, no GPS API calls, no upload bucket calls. Each concern lives one layer below, swappable and testable.

```
Screen
  → useRequest()           ← hook (what screens call)
    → request.repository.ts ← only file that knows Supabase exists
      → Supabase           ← swap this file to change backends

# Example screen (describe.tsx):
const { location } = useLocation()
const { control, handleSubmit } = useForm<RequestFormData>()
const { mutate: createRequest, isPending } = useCreateRequest()
# Screen has zero knowledge of GPS APIs or Supabase.
```

Folder Structure

Every directory and its purpose:

app/	Expo Router file-based screens. Grouped by role.
app/(auth)/	Login and onboarding screens
app/(customer)/	All 6 customer flow screens
app/(worker)/	Worker dashboard and job detail screens
features/	Feature-first logic. Each feature is self-contained.
features/auth/	auth.repository · auth.store · useAuth · auth.types
features/request/	request.repository · request.store · useRequest · types
features/worker/	worker.repository · worker.store · useWorker · types
features/location/	useLocation hook — GPS only, no repo needed
features/notifications/	notifications.service · useNotifications
components/ui/	Atomic: Button, Input, Badge, Avatar
components/layout/	Screen, SafeArea, KeyboardAvoider
components/liquid/	Animated: LiquidCard, SlideSheet, PulseRing
components/map/	MapView wrapper, Marker, ETA overlay
design/	Design system: tokens, typography, motion, shadows
lib/	supabase.ts (single client), constants.ts
hooks/	Cross-feature shared hooks: useRealtimeSubscription, useDebounce
types/	supabase.generated.ts, api.types.ts, navigation.types.ts

SECTION 4

Database Schema

All tables are hosted on Supabase (PostgreSQL). Row-Level Security is enabled on every table from day one. Never skip this step.

profiles (extends supabase auth.users)		
Field	Type	Notes
id	uuid PK	references auth.users – cascades on delete
full_name	text NOT NULL	
phone	text UNIQUE	
avatar_url	text	optional profile photo
role	text	customer worker admin
fcm_token	text	push notification token
created_at	timestampz	default now()

workers (extends profiles)		
Field	Type	Notes
id	uuid PK	references profiles(id)
skills	text[]	['plumbing', 'electrical', 'general']
bio	text	optional worker description
rating	numeric(2,1)	aggregated from reviews
total_jobs	int	completed job count
is_available	boolean	worker toggles this themselves
current_lat	float8	live location – updated every 60s
current_lng	float8	live location – updated every 60s
verified	boolean	admin manually verifies in Phase 1
id_doc_url	text	KYC document upload – future compliance

categories (admin-managed, not hardcoded)		
Field	Type	Notes
id	uuid PK	gen_random_uuid()
name	text UNIQUE	e.g. 'Plumbing', 'Electrical'
icon_name	text	maps to icon library key
description	text	

<code>is_active</code>	boolean	admin can disable categories
<code>sort_order</code>	int	display order in app

requests (core transaction table)

Field	Type	Notes
<code>id</code>	uuid PK	gen_random_uuid()
<code>customer_id</code>	uuid FK	references profiles(id)
<code>worker_id</code>	uuid FK	null until matched
<code>category_id</code>	uuid FK	references categories(id)
<code>description</code>	text	optional issue description
<code>photo_urls</code>	text[]	Supabase Storage URLs
<code>lat / lng</code>	float8	customer GPS at time of request
<code>address_label</code>	text	reverse-geocoded human address
<code>service_type</code>	text	home roadside (future-proofed)
<code>status</code>	text	pending searching matched en_route in_progress completed can
<code>matched_at</code>	timestampz	
<code>completed_at</code>	timestampz	
<code>customer_rating</code>	int	1-5 stars
<code>customer_review</code>	text	
<code>vehicle_id</code>	uuid FK	null – populated when roadside launches
<code>created_at</code>	timestampz	

transactions (payment-ready from day one)

Field	Type	Notes
<code>id</code>	uuid PK	
<code>request_id</code>	uuid FK	references requests(id)
<code>amount</code>	numeric(10,2)	in GHS
<code>platform</code>	text	mtn_momo vodafone_cash airteltigo cash
<code>platform_ref</code>	text	payment gateway transaction ID
<code>status</code>	text	pending paid failed refunded
<code>commission_pct</code>	numeric(4,2)	default 15.00 – your cut
<code>worker_payout</code>	numeric(10,2)	calculated at completion

RLS Policies — apply these before any user touches the app

Customers: SELECT own requests only (`auth.uid() = customer_id`). Workers: SELECT requests matched to them (`auth.uid() = work`)

SECTION 5

Design System

The liquid iOS feel comes from three things: spring physics, material blur, and haptics. Not from gradients. Never from gradients.

Design Tokens — design/tokens.ts

```
export const tokens = {
  colors: {
    primary:      '#0052CC',
    surface:      'rgba(255,255,255,0.85)', // liquid glass
    surfaceDark:  'rgba(28,28,30,0.85)',
    border:       'rgba(0,0,0,0.08)',
    danger:       '#FF3B30',
    success:      '#34C759',
    warning:      '#FF9F0A',
  },
  radius: { sm: 10, md: 14, lg: 20, xl: 28, full: 9999 },
  spacing: { xs: 4, sm: 8, md: 16, lg: 24, xl: 32, xxl: 48 },
} as const
```

Motion Configs — design/motion.ts

```
export const springs = {
  snappy: { damping: 18, stiffness: 280 }, // button presses, toggles
  bouncy: { damping: 14, stiffness: 200, mass: 0.8 }, // cards entering
  smooth: { damping: 22, stiffness: 180 }, // page transitions
}
```

Component Library

LiquidCard	Spring scale on press + BlurView surface. Core reusable card.
SlideSheet	@gorhom/bottom-sheet wrapper — match found, worker detail
PulseRing	Animated ring for searching screen — the "Uber magic moment"
Button	Reanimated scale + expo-haptics on every press
Screen	SafeAreaView + KeyboardAvoidingView + background wrapper
MapView	react-native-maps wrapper with custom marker + ETA overlay
PulseRing	Looping animated circle — searching / live tracking indicator

Android blur — use with caution

BlurView renders natively on iOS. On Android below API 31 it falls back to a semi-transparent solid. Test LiquidCard on a sub-4GB.

SECTION 6

App Flow & Screen Map

Customer Flow — 6 Screens

Screen 1 — Entry (`index.tsx`)

One LiquidCard, one "Get Help Now" button, one line of subtext. No menus. No clutter. No philosophy.

- Categories loaded from Supabase on mount
- Auth guard redirects to login if no session
- Haptic on button press

Screen 2 — Select Problem (`select-problem.tsx`)

Grid of category cards loaded dynamically from Supabase. 3–5 categories max in Phase 1.

- categories fetched from DB — not hardcoded
- Tapping a card navigates with `category_id` param
- Disabled state for `is_active = false` categories

Screen 3 — Describe Issue (`describe.tsx`)

Auto-filled GPS location, optional text description, optional photo upload. One "Request Help" button.

- expo-location auto-fills lat/lng + reverse-geocoded address
- expo-image-picker for photo — stored to Supabase Storage
- react-hook-form + zod validates before submit
- GPS denied → show manual address text input fallback

Screen 4 — Searching (`searching.tsx`)

The "Uber magic moment". PulseRing animation. "Finding nearby handyman..." text. Supabase realtime listens for status change.

- `subscribeToRequest()` fires on status change in DB
- Admin manually updates `request.status = matched` in Phase 1
- Timeout after 15 minutes → show "No workers available" state
- Cancel button available throughout

Screen 5 — Match Found (`match.tsx`)

Worker name, skill badge, ETA. Map showing worker approximate location. One-tap phone call button.

- Worker data pulled from matched `request.worker_id`
- Phone button: `Linking.openURL("tel:+233...")`
- Map initialises on worker `current_lat/current_lng`
- ETA shown as static field in Phase 1 — automated in Phase 2

Screen 6 — Review (review.tsx)

Was the issue fixed? Yes / No. 1–5 star rating. Optional text.

- Updates request.status = completed
 - Writes customer_rating + customer_review to requests table
 - Worker rating recalculated async
-

Worker Flow

Worker screens are intentionally minimal in Phase 1. The worker toggle is the most critical interaction — it determines whether they appear in the matching pool.

Incoming Jobs (worker/index.tsx)

List of incoming requests. Toggle availability on/off. Accept / Decline each job.

Job Detail (worker/job-detail.tsx)

Category, description, photo, customer address on map. Accept → updates request status. Call customer button.

SECTION 7

Implementation Plan

Phase 1 — MVP (Weeks 1–4)

FOUNDATION

Sprint 1 — Project Setup

Week 1

- ☐ Init Expo project with TypeScript template
- ☐ Configure ESLint, Prettier, and absolute import paths
- ☐ Install expo-router and create full folder structure
- ☐ Push to GitHub — create main and dev branches
- ☐ Install all Phase 1 libraries (3 batches)
- ☐ Configure react-native-reanimated in babel.config.js
- ☐ Create Supabase project, save keys to .env
- ☐ Write lib/supabase.ts — single client instance
- ☐ Run full DB schema migration (all 5 tables)
- ☐ Enable RLS policies on every table
- ☐ Generate supabase.generated.ts via Supabase CLI

FOUNDATION

Sprint 2 — Design System

Week 1–2

- ☐ Write design/tokens.ts — colors, spacing, radius, z-index
- ☐ Write design/typography.ts — font scale
- ☐ Write design/motion.ts — spring configs (snappy, bouncy, smooth)
- ☐ Configure NativeWind with token values
- ☐ Build components/layout/Screen.tsx — safe area wrapper
- ☐ Build components/ui/Button.tsx — with haptic feedback
- ☐ Build components/ui/Input.tsx and Badge.tsx
- ☐ Build components/liquid/LiquidCard.tsx — spring scale + blur
- ☐ Build components/liquid/SlideSheet.tsx — bottom sheet wrapper
- ☐ Build components/liquid/PulseRing.tsx — for searching screen
- ☐ Test LiquidCard blur on mid-range Android device

CORE

Sprint 3 — Auth Flow

Week 2

- ☐ Write features/auth/auth.types.ts
- ☐ Write features/auth/auth.repository.ts (signUp, signIn, signOut, getSession)
- ☐ Write features/auth/auth.store.ts — zustand slice
- ☐ Write features/auth/useAuth.ts
- ☐ Build app/(auth)/login.tsx — phone number + OTP
- ☐ Build app/(auth)/onboarding.tsx — name + role selection
- ☐ Add auth guard to root _layout.tsx
- ☐ End-to-end test: register, login, logout on physical device

CORE**Sprint 4 — Customer Request Flow**

Week 2–3

- ☐ Write features/location/useLocation.ts — permissions + watch
- ☐ Add reverse geocoding to turn lat/lng into address string
- ☐ Write features/request/request.types.ts
- ☐ Write features/request/request.repository.ts (create, getById, subscribe)
- ☐ Write features/request/request.store.ts and useRequest.ts
- ☐ Build index.tsx — entry screen with LiquidCard
- ☐ Build select-problem.tsx — dynamic category cards
- ☐ Build describe.tsx — GPS, text, photo upload
- ☐ Build searching.tsx — PulseRing + realtime subscription
- ☐ Build match.tsx — worker info, map, call button
- ☐ Build review.tsx — rating and completion

INFRA**Sprint 5 — Admin + Worker Ops**

Week 3

- ☐ Write features/worker/worker.repository.ts
- ☐ Write features/worker/useWorker.ts
- ☐ Build app/(worker)/index.tsx — jobs list + availability toggle
- ☐ Build app/(worker)/job-detail.tsx — accept / decline
- ☐ Write features/notifications/notifications.service.ts
- ☐ Store FCM token to worker profile on login
- ☐ Supabase Edge Function: send push on job matched
- ☐ Seed 3–5 test workers in DB
- ☐ Verify Supabase dashboard usable for manual dispatch
- ☐ Run full end-to-end manual dispatch test

HARDENING**Sprint 6 — Africa Mode + QA**

Week 4

- ☐ Write hooks/useAppState.ts — foreground / background detection
- ☐ Add exponential backoff retry (3 attempts) to all repo calls
- ☐ Offline banner — visible when NetInfo reports no connection
- ☐ Queue failed requests in AsyncStorage — retry on reconnect
- ☐ GPS denied fallback — manual address text input
- ☐ No workers available state — graceful UI message
- ☐ Request timeout at 15 minutes with cancellation flow
- ☐ Add error boundary to root layout
- ☐ Test on low-end Android (4GB RAM or less)
- ☐ Test on throttled 3G connection via dev tools
- ☐ Build APK via EAS Build and install on real device
- ☐ Get 3 real people to submit a test request end-to-end

Phase 2 — Automate (Months 2–6)

CORE

Sprint 7 — Auto-Matching Engine

Month 2

- ☐ Supabase Edge Function: match_request
- ☐ Query available workers by skill, sort by Haversine distance
- ☐ Broadcast to top 3 nearest workers simultaneously
- ☐ First-accept-wins with atomic status update
- ☐ Fallback to next 3 workers if no response in 2 minutes
- ☐ Worker background location update every 60s when available
- ☐ Update workers.current_lat/long on each location ping

INFRA

Sprint 8 — Admin Dashboard

Month 2–3

- ☐ Scaffold Next.js admin app (separate repository)
- ☐ Live requests board via Supabase realtime
- ☐ Worker management — verify, suspend, view job history
- ☐ Analytics: requests/day, avg response time, completion rate
- ☐ Admin action log audit trail UI

INFRA

Sprint 9 — Mobile Money Integration

Month 3

- ☐ Register with Paystack (supports MTN MoMo, Vodafone Cash, AirtelTigo)
- ☐ Spin up Node.js backend service for payment webhooks only
- ☐ Populate transactions table on successful payment event
- ☐ Worker payout calculation — net after commission
- ☐ Failed payment retry flow

Phase 3 — Platform (Month 6+)

PLATFORM

Sprint 10 — Roadside Vertical

Month 6+

- ☐ Add vehicles table to DB (make, model, plate, insurance_expiry)
- ☐ Add roadside categories (Mechanic, Towing, Tire, Battery, Fuel)
- ☐ Entry point — home vs roadside selection on open
- ☐ Roadside describe screen — includes vehicle info
- ☐ Roadside workers have vehicle breakdown skills in their profile

PLATFORM

Sprint 11 — Scale + Multi-City

Month 9+

- ☐ Migrate hot matching logic from Edge Functions to Node.js backend
- ☐ Add BullMQ / Redis queue for request processing under spike load
- ☐ Replace Supabase realtime with Socket.IO for multi-server scale
- ☐ Add city_id to workers and requests tables for multi-city support
- ☐ Subscription model for workers — recurring MTN MoMo payment
- ☐ Fleet / corporate contract dashboard in admin app

SECTION 8

Worker Onboarding Strategy

Getting supply right is more important than any feature. BOA dies if a customer requests help and nobody responds. The following strategy is layered — deploy in order, do not skip to Phase 3 because it sounds impressive.

Zone Agent Model — Deploy First [Month 1]

Recruit 3–5 trusted, smartphone-owning handypeople you personally know. Give them full worker app access. They receive job requests.

Hardware Store Registration Points [Month 1]

Handypeople buy supplies at the same stores repeatedly — Buildmart, iron-benders, electrical wholesalers, paint shops. Place a "BOA Worker" sign.

WhatsApp Business API — For Low-End Android [Month 2]

Workers who have WhatsApp but will not download another app can receive jobs via WhatsApp message. Job comes in → message → worker.

USSD Self-Registration — For Feature Phones [Month 3]

Lease a USSD shortcode from MTN (~\$200–500/month). Workers dial *384*BOA# on any phone — no smartphone, no internet required.

Trade Association Partnership [Month 3]

Approach Ghana Electrical Contractors Association, Ghana Institute of Plumbers, Suame Magazine artisan cooperatives. Propose: "We'll help you get more work."

Community Radio — Twi Spots [Month 2]

Nhyira FM, Oyerepa FM, Kumasi FM. A 30-second Twi spot reaches exactly the demographic you need at very low cost. "Wo ye plimfo!"

The Golden Rule — treat first 10 workers like kings

Word spreads fast in Kumasi. One worker who got cheated tells 20 others. One worker paid same-day via MoMo tells 20 others too.

SECTION 9

Africa Mode — Network Resilience

Do not build a "perfect app" that fails in real conditions. Ghana has intermittent 2G/3G in many areas. Mid-range Android is the target device. Every critical flow must have a fallback.

Retry logic on all repo calls

Exponential backoff — 500ms, 1s, 2s — max 3 attempts before showing error state. Applies to every Supabase call in every repository file.

Offline banner

useNetInfo() from @react-native-community/netinfo detects loss of connectivity. Show a persistent banner: "You are offline. Reconnecting..." Do not crash.

Request queue — AsyncStorage

If createRequest() fails due to network, store the payload in AsyncStorage. On reconnect (useAppState detects foreground return), retry queued requests.

GPS permission denied fallback

If user denies location permission, show a manual text input for address. Never block the user from requesting help due to GPS failure.

No workers available — graceful state

If realtime produces no match after 15 minutes, show a friendly message and offer to retry or cancel. Never leave the user staring at a spinner forever.

Image upload — compress before sending

Use expo-image-picker with quality: 0.6 and maxWidth: 1200 to reduce payload size. A 6MB photo fails on 2G. A 400KB compressed photo does not.

Low-end Android test — non-negotiable

Before shipping to any real user, install the APK on a device with 4GB RAM or less. Reanimated animations that are smooth on a Pixel 8 can jank hard on a Tecno Spark.

SECTION 10

Growth Roadmap & Monetisation

Monetisation Model

BOA takes a commission on every completed job. No subscription for customers. Workers pay nothing in Phase 1 — this is how you build supply density fast.

Phase 1	Commission model	15% of job value, collected as "cash" transaction (manual)
Phase 2	Mobile Money	Paystack processes payment, commission auto-deducted
Phase 3	Worker subscription	Monthly fee for priority matching + verified badge
Phase 3	Corporate contracts	Flat monthly fee for businesses with maintenance needs
Phase 3	Insurance integration	Referral or white-label roadside coverage

Expansion Verticals — In Order

1 — Emergency Handyman (NOW)	Plumbing, electrical, general repair in Kumasi.
2 — Roadside Rescue (Month 6)	Mechanics, towing, tire repair, battery, fuel.
3 — Motorbike Emergency (Month 9)	Huge segment in Ghana — riders are underserved.
4 — Multi-City: Accra (Month 12)	Same playbook — zone agents first, then USSD.
5 — East Africa (Year 2)	Nigeria, Kenya — partnership or franchise model.

The Launch Test — Your Only Real KPI

One metric that matters in Phase 1

Can you get 5 real people to submit a request and have a worker show up? If yes — you are onto something. If no — do not write a

Future Additions — futureAdditions.md

Keep this list in a futureAdditions.md at repo root. Revisit it at the start of each new phase.

- Node.js custom backend — start with payment webhooks only
- BullMQ / Redis queue — add when volume exceeds Supabase Edge Function limits
- Socket.IO — replace Supabase realtime when multi-server scale is needed
- USSD shortcode (*384*BOA#) — worker self-registration without smartphone
- WhatsApp Business API — job dispatch for non-app workers
- CI/CD pipeline — GitHub Actions → EAS Build on merge to main
- Offline-first with full sync — AsyncStorage + background sync loop
- SMS fallback via Hubtel or Twilio when push notification fails

- Insurance partnership API integration
- Fleet / corporate contract admin dashboard
- @tanstack/react-query — add when data fetching gets complex (Phase 2)
- Tamagui — evaluate if NativeWind performance issues appear on Android
- In-app chat between customer and worker
- Worker earnings dashboard — total jobs, total earned, pending payout

End of Document

BOA — Build it. Ship it. Iterate.